

Baobab

Rooted here. Reaching there.

Business and Technical Specification

ECO5040W Project, Group 3

Karabo Tigedi Kerry-Lynn Whyte Nikola Milosavljevic
Annita Ngoma Lilitha Mzamo

Contents

1 Business Problem	1
2 User Stories	1
2.1 Sender	1
2.2 Receiver	1
3 Competitor Comparison	1
3.1 Conventional money transfer operators	2
3.2 VALR Pay	2
3.3 Wise	3
3.4 Summary	3
4 User Journey	3
4.1 Onboarding	4
4.2 Cash-in and settlement	4
4.3 Wallet and cash-out	5
5 Functional Requirements	5
5.1 Registration and authentication	5
5.2 KYC	5
5.3 Beneficiary management	6
5.4 Quotes, fees and limits	6
5.5 Simulated cash-in	6
5.6 Queue and settlement	6
5.7 Recipient wallet and sender history	7
5.8 Simulated cash-out	7
5.9 Administration	7
6 Fee Model	7
7 Exchange-Rate Calculation	8
8 Remittance Limits	8
9 Architecture	10
10 Cash-in Flow	11
11 RLUSD Settlement Flow	11
12 Cash-out Flow	12
13 Database Design	13
14 API Overview	14
15 Security Design	14
16 Regulatory Considerations	14
16.1 KYC and AML	14
16.2 Licensing	15
16.3 Consumer protection	15
16.4 Custody and safeguarding of customer assets	15
16.5 In the prototype	15
17 Assumptions and Limitations	16
References	16

1 Business Problem

Sending money from South Africa to neighbouring countries remains expensive, particularly in the SA to Zimbabwe corridor, which is one of the country's largest remittance routes (FinMark Trust, 2024). The World Bank's Remittance Prices Worldwide database records what major providers charge for its USD 200 benchmark transfer, combining the transfer fee and the exchange-rate margin into a single total cost. In data collected in August 2025, that cost ranged from 5.15% through WorldRemit's online service to 31.81% through Western Union's agent network, with MoneyGram at 15.43% and Mukuru at 10.28% (World Bank, 2025). The corridor average was 12.49%, more than four times the Sustainable Development Goal target of 3%, and even the cheapest option exceeded the goal's separate aim of eliminating corridors that cost more than 5% (United Nations, 2015).

These costs fall on the migrant workers who rely on this corridor and others like it. About R19 billion was sent formally from South Africa to other Southern African Development Community (SADC) countries in 2024, nearly 90% of it to Lesotho, Zimbabwe, Mozambique and Malawi (FinMark Trust, 2024). Most of that these transactions are small, regular amounts which are set aside from senders' wages, but these amounts are often consumed by fixed fees (FinMark Trust, 2024).

Stablecoin platforms provide a way to circumvent this. Because a dollar-pegged token such as USDC or USDT can move directly between accounts, the chain of correspondent banks is removed, and value can be transferred almost instantly and at little cost (International Monetary Fund, 2025). Many of these platforms, however, are designed primarily for people who already use cryptocurrency, and they leave both ends of the transfer to the user. The sender must first buy the stablecoin with rand before sending it, and the recipient must hold it as a crypto asset and then sell it to obtain cash (Rey, 2025). For a migrant worker and a family at home who need the money to arrive, each of these steps adds cost, effort and room for error. Stablecoins make the transfer itself cheap; on their own, however, they do not amount to a remittance service.

Baobab is built to turn stablecoin settlement into a complete remittance service. Senders pay in rand, just as they would with MoneyGram, Western Union or Mukuru, while the transfer itself settles on the XRP Ledger in RLUSD, giving the transfer the cost and speed of a stablecoin. Neither party has to handle the stablecoin directly: the platform converts the rand on the sender's behalf, and a custodial wallet, in which the platform holds the private keys on the recipient's behalf, lets the recipient keep the funds in RLUSD or cash out to US dollars or local currency. In a production system, that cash-out would rely on local payout partners able to convert RLUSD into local currency. In the prototype, this step is simulated, and RLUSD is represented by UCTUSD, a course-issued test token on the XRP Ledger Testnet that is treated as being worth one US dollar.

2 User Stories

2.1 Sender

As a migrant worker in South Africa, I want to send part of my wages to my family in Zimbabwe at a lower cost and it needs to arrive on the same day, so that more of my money reaches them and they are not left waiting days for the money they depend on.

2.2 Receiver

As a family member in Zimbabwe who depends on money sent from South Africa, I want to receive it as dollar value I can keep or cash out, without having to manage or sell cryptocurrency myself, so that a cheaper transfer does not come at the cost of more difficulty for me.

3 Competitor Comparison

The comparison focuses on four conventional money transfer operators on the South Africa to Zimbabwe corridor: the global incumbents MoneyGram and Western Union, Mukuru, whose business is built around the migrant-remittance pattern described in Section 1, and WorldRemit, the cheapest digital provider in the World Bank's data. VALR Pay is also looked at as a stablecoin-native provider, and Wise as a low-cost digital benchmark. While not extensive, it paints a picture of the current market landscape.

3.1 Conventional money transfer operators

All four operators are compared using the World Bank’s Remittance Prices Worldwide data collected between 15 and 26 August 2025, which provides independently collected, directly comparable pricing from a single non-commercial source. Each figure is the total cost of sending USD 200, the World Bank’s benchmark amount.

	MoneyGram	Western Union	Mukuru	WorldRemit
Channel	Agent; cash pickup	Agent; cash pickup	Call centre; cash pickup	Online; mobile wallet
Total cost	15.43%	31.81%	10.28%	5.15%
Cost structure	Flat fee plus 0.83% exchange-rate margin	Flat fee; exchange rate marginally favourable	Flat fee plus 0.35% exchange-rate margin	Flat fee plus 3.15% exchange-rate margin
Speed	Next day	Two days	Same day	Under one hour
Pain points	Cost roughly five times the SDG target; slower than digital providers	Highest cost of the four; price varies widely by channel (15.85% via Absa online banking)	Twice the cost of the cheapest digital providers; customers report limited collection points, strict name-matching at payout and slow dispute resolution	Lowest cost, but available only online by bank transfer or card, excluding senders without either

Table 1: Source: World Bank (2025), Remittance Prices Worldwide, South Africa to Zimbabwe corridor, Q3 2025. Total cost combines the fee and the exchange-rate margin as a percentage of the amount sent, while speeds are as recorded in the same database.

The comparison reveals a market in which price bears very little relation to the rendered service. Western Union’s agent channel charges around six times as much as WorldRemit yet takes two days to deliver, while money sent through WorldRemit arrives in under an hour. On the World Bank’s USD 200 benchmark, that gap amounts to roughly USD 53 per transfer, a considerable sum for a worker sending money home every month. The digital providers have brought costs much closer to the target, but even the cheapest remains above 3%, leaving room for a model that can reduce costs further.

3.2 VALR Pay

VALR Pay is a direct stablecoin competitor on this corridor, and it addresses the same problem as the proposed platform. VALR, a South African crypto-asset exchange with over 1.8 million users, frames the corridor in the same terms as this analysis: it identifies Western Union and Mukuru as the familiar options for sending money to Zimbabwe, and argues that these platform’s fees and exchange rates make them costly for people who send regularly, while many recipients lack the bank accounts that cheaper transfers require (VALR, 2025a). VALR suggests to move the money as a stablecoin such as USDC or USDT, instantly and without a transfer fee (VALR, 2025a). In doing so, VALR shows that stablecoin settlement can remove the cost of the transfer itself. What it does not remove is the cost of converting into and out of the stablecoin, and that is where it falls short of a complete remittance service.

- **Access:** both sender and recipient need a VALR account, and opening one requires identity verification, including proof of a residential address. Payments to accounts that are not fully verified are reversed within 72 hours.
- **Pricing:** the transfer itself is free, but the steps on either side are not. To send dollar value rather than rand, the sender must first convert rand into a stablecoin, for which VALR’s Simple Buy service charges a conversion fee of 1.6%. A recipient who moves the stablecoin off the platform pays a network-dependent withdrawal fee, which VALR quotes at the time of withdrawal and which may exceed the network’s actual charge. The cost of then converting the stablecoin into cash is set by whichever exchange or agent the recipient uses, and is not known in advance.
- **Speed:** instant between VALR accounts, since the transfer is recorded on VALR’s own platform rather than passed through correspondent banks. The time taken to reach cash depends on the steps the recipient takes afterwards.
- **Pain points:** VALR’s fiat services are built around the rand. Its help centre notes that deposits in other

currencies are credited as a stablecoin, with ZAR the only currency it credits directly (VALR, n.d.-a), and no option was found to withdraw directly to a Zimbabwean bank account or mobile wallet. A recipient in Zimbabwe who needs cash must therefore hold the stablecoin, move it to another platform and sell it there, taking on the associated fees and risks.

VALR's stablecoin infrastructure is also beginning to reach customers through the conventional operators themselves. In November 2025, VALR and Mukuru launched a USDC wallet in South Africa, offered through Mukuru's WhatsApp platform, which allows Mukuru's customers to buy, hold and sell USDC. The service is presented as a way to save in dollars rather than as a cross-border transfer, but it shows a major operator on this corridor adopting the same stablecoin infrastructure that underpins VALR Pay.

3.3 Wise

Wise is a significant new entrant into South Africa's outbound market. The company serves over 18.9 million customers worldwide and converts money at the mid-market exchange rate, charging a separately disclosed fee rather than building its margin into the rate (Wise, 2025, 2026). In December 2025 it received conditional approval from the South African Reserve Bank to operate as a Category 2 Authorised Dealer in Foreign Exchange with Limited Authority, its first regulatory approval in Africa, and in April 2026 its infrastructure began powering international transfers for Capitec, a bank with more than 25 million customers. That combination gives Wise almost immediate reach into a large share of South Africa's banked population. On this corridor, however, its presence is limited. Wise does not appear among the providers surveyed on the South Africa to Zimbabwe route (World Bank, 2025), and its South African service runs through bank accounts, whereas several of the corridor's established providers are built around cash collection. Its licence and Capitec's customer base give Wise the infrastructure to extend its model to regional corridors, and if it did so on this route, its pricing would make it a direct competitor on cost. Wise therefore sets the standard for price transparency that any competitor on this corridor must meet, but it does not yet serve the cash-based remittances that migrant workers rely on.

3.4 Summary

Taken together, the comparison shows a market split three ways. The global incumbents offer wide cash networks at 15% to 32% of the amount sent. Digital providers such as WorldRemit have brought costs down to around 5%, with delivery in under an hour, yet none reaches the 3% target. VALR Pay removes the transfer fee but leaves buying, holding and selling the stablecoin to its users. The platform competes on this ground. It delivers a complete service from rand to cash, and the recipient can hold the funds as dollar value rather than cash out immediately. At its default fees (R25 plus 1%, a 2% exchange-rate margin and a 1.5% cash-out fee) the total cost is about 5.2% at the R3,000 daily cap and higher on smaller amounts. That is close to WorldRemit but not below it, and not under the 3% target. The fees are configurable by an administrator, so they can be lowered in future without a code change.

4 User Journey

The journey followed by the senders and recipients can be seen in the sequence diagrams below.

4.1 Onboarding

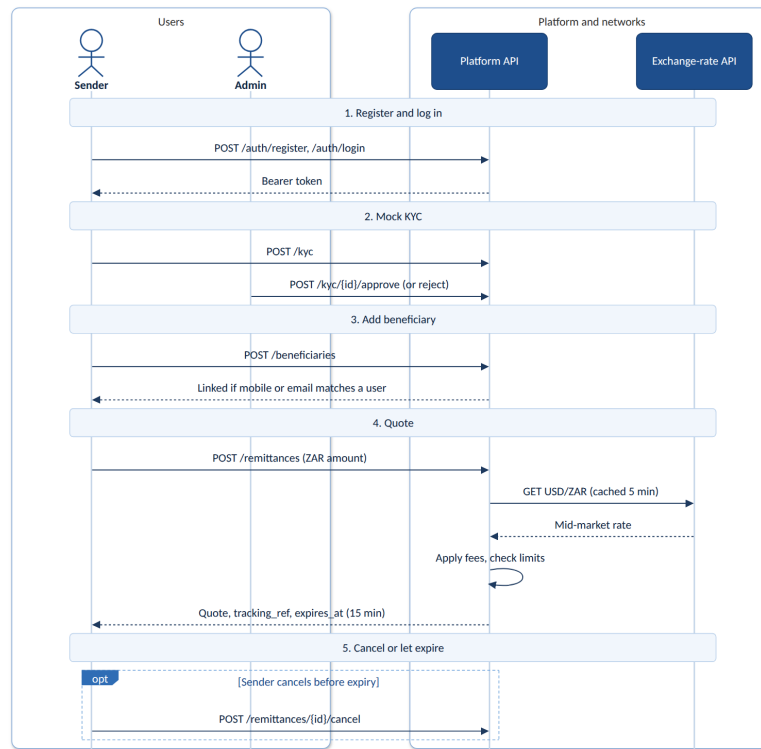


Figure 1: Register, KYC, beneficiary, quote and cancel.

4.2 Cash-in and settlement

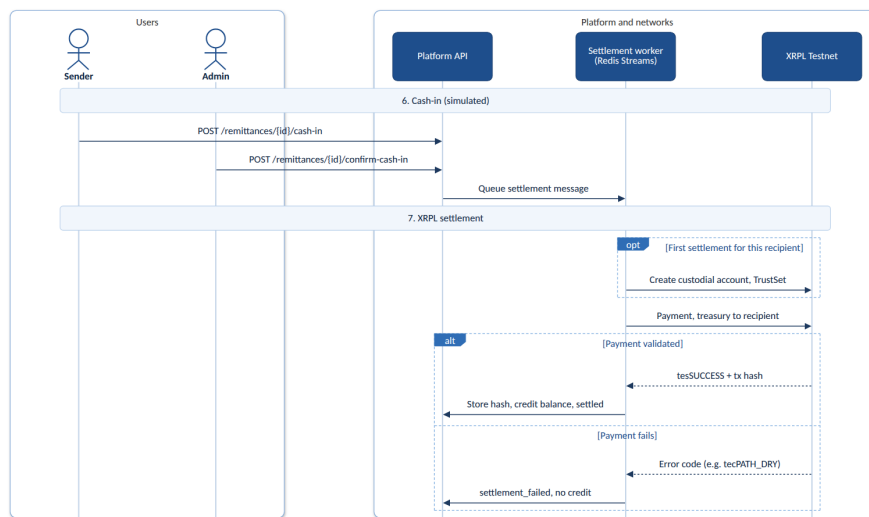


Figure 2: Cash-in confirmation and XRPL settlement.

4.3 Wallet and cash-out

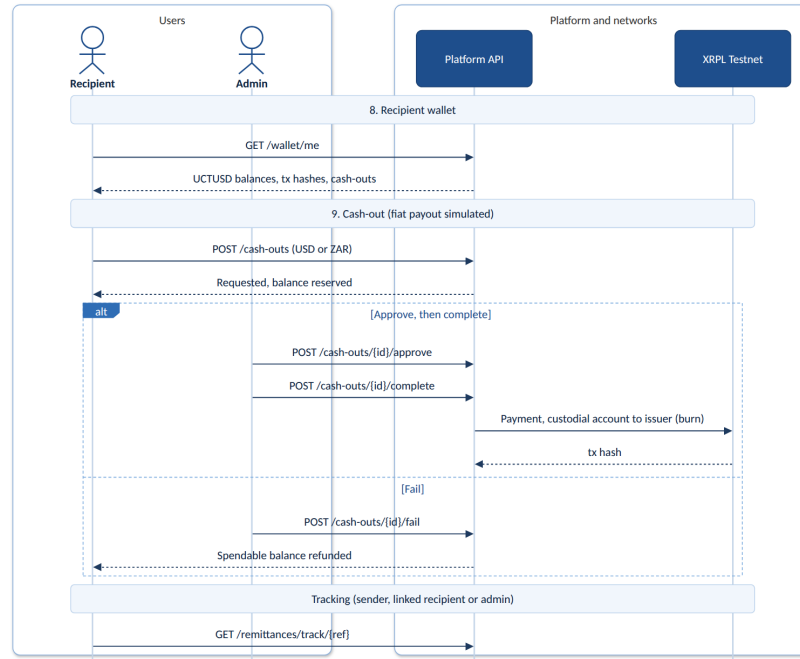


Figure 3: Recipient wallet, cash-out and tracking.

5 Functional Requirements

5.1 Registration and authentication

ID	Requirement	Implemented in
FR-01	Register an account with personal details (full name, email address, mobile number and a password).	POST /auth/register; UI /register. Mobile numbers must be 8–15 digits with an optional leading + (ensures complying with international standards); passwords are bcrypt-hashed.
FR-02	Log in with registered credentials.	POST /auth/login returns a bearer token (only its SHA-256 hash is stored) and logs the user in.
FR-02a	Log out, terminating the session.	POST /auth/logout sets revoked_at on the session and logs the user out.
FR-03	Email address and mobile number are unique across accounts.	Unique constraints on users.email and users.mobile_number ensure that a duplicate registration returns a 409 error (not allowed).
FR-04	Distinguish sender, recipient and administrator roles.	Partial, by design. Stored roles are customer and admin; sender and recipient are what a customer does on a given transfer. Admins come only from scripts/create_admin.py or the ADMIN_EMAIL/ADMIN_PASSWORD start-up setting.
FR-04a	View and update basic profile information.	GET/PATCH /users/me; UI /app/profile. Allows the user to update profile info

5.2 KYC

ID	Requirement	Implemented in
FR-05	Collect full name, date of birth, nationality, identification number, residential address, mobile number, email address and source of funds.	POST /kyc stores all eight fields. The ID number (6–20 characters) is encrypted with the KYC Fernet key.
FR-06	Submit a completed KYC application for review.	POST /kyc overwrites a pending or rejected application and returns 409 once it is approved. The screen locks the form while it is pending.
FR-07	Administrator can view a submitted application.	GET /kyc, GET /kyc/{id}; UI /admin/kyc.
FR-08	Administrator can approve or reject an application.	POST /kyc/{id}/approve or /reject. A missing or blank rejection reason returns 422; the UI asks for one and confirms the decision in a dialog naming the applicant.
FR-08a	User can view their own KYC status.	GET /kyc/me/status, GET /kyc/me; shown on the sender home screen.
FR-09	No remittance without approved KYC.	POST /remittances returns 403 without approved KYC; the send screen shows “KYC not approved” with a line for pending, rejected or not submitted.
FR-09a	No cash-out without approved KYC.	POST /cash-outs requires approved KYC. A recipient can still receive and hold UCTUSD without it.

The assumption attached to FR-09a is adopted as stated: senders need approved KYC before sending and recipients before cashing out, but receiving and holding does not require KYC approval.

5.3 Beneficiary management

ID	Requirement	Implemented in
FR-10	Approved sender adds a beneficiary with full name, mobile number or email, country, payout currency and relationship.	POST <code>/beneficiaries</code> , approved senders only; all five fields stored. UI <code>/app/beneficiaries</code> . The <code>payout_currency</code> is stored and shown, but quotes do not use it.
FR-11	Require at least a mobile number or an email address.	Request validation rejects a beneficiary with neither (422).
FR-12	Sender can list their beneficiaries.	GET <code>/beneficiaries</code> , each with a Linked or Unlinked badge in the UI.
FR-12a	Link a beneficiary to a recipient account by matching mobile number or email.	Auto-match when the beneficiary is added, and again whenever a new account registers (<code>services/beneficiary_linking.py</code>). Mobile or email is enough; the first matching account wins. Editing a profile does not relink. Email matching is case-insensitive.
FR-12b	Provision a custodial wallet only once the beneficiary is linked to a registered recipient account.	A wallet row is created when the beneficiary is linked; the recipient's custodial Testnet account is created at the first settlement, and only for the linked user.
FR-12c	If no account matches, leave the beneficiary unlinked; no wallet and no settlement until a matching account registers.	The record stays unlinked. If cash-in is confirmed anyway, the worker refuses to settle ("Beneficiary is not linked to a registered recipient account"), creates no wallet, credits nothing, and marks the transfer <code>settlement_failed</code> ; an administrator retries once the recipient registers and is linked.

5.4 Quotes, fees and limits

ID	Requirement	Implemented in
FR-13	Determine the USD/ZAR rate when the transaction is created (public API, mock or rate table).	<code>open.er-api.com</code> at quote creation, cached for five minutes, with a <code>.env</code> fallback rate (Section 7). The rate used is stored on the quote.
FR-14	Quote shows ZAR amount, rate, transaction fee, FX margin, RLUSD amount, cash-out fee and estimated payout.	POST <code>/remittances</code> returns <code>zar_amount</code> , <code>exchange_rate</code> , <code>transaction_fee_zar</code> , <code>fx_margin_percentage</code> , <code>rlusd_amount</code> , <code>estimated_cash_out_fee</code> and <code>estimated_recipient_payout</code> , plus <code>tracking_ref</code> and <code>expires_at</code> . The <code>estimated_recipient_payout</code> is in UCTUSD, not rand, and is an estimate.
FR-15	Fees use a configurable fixed fee, percentage fee, FX margin and cash-out fee.	One <code>FeeConfig</code> row (Section 6).
FR-15a	Administrator can view and update the fee parameters.	GET/PUT <code>/admin/fee-config</code> ; UI <code>/admin/config</code> .
FR-16	Check the amount against daily and monthly limits before proceeding.	Checked on the gross rand amount before the quote is inserted; refused only when usage plus the amount is strictly over the cap. Cancelled and expired quotes do not count towards the limit. Two simultaneous quotes are not locked, so both can pass.
FR-16a	Sender can view limits and remaining allowance.	GET <code>/limits/me</code> returns limits, usage and remaining amounts for today and this month.
FR-16b	Administrator can view and update limit tiers.	GET <code>/admin/limit-tiers</code> , PUT <code>/admin/limit-tiers/{tier_key}</code> ; UI <code>/admin/config</code> .
FR-17	Reject a request that would exceed a limit, and say why.	HTTP 422 naming the limit, e.g. "This remittance would exceed your daily limit of ... ZAR".

5.5 Simulated cash-in

ID	Requirement	Implemented in
FR-18	Sender initiates a simulated ZAR payment by a supported method.	POST <code>/remittances/{id}/cash-in</code> with <code>agent_cash</code> , <code>bank_transfer</code> or <code>card</code> ; only for an unexpired, uncanceled quote.
FR-19	Administrator confirms the payment.	POST <code>/remittances/{id}/confirm-cash-in</code> ; UI <code>/admin/cash-in</code> . There is no mock payment-provider route; the administrator is the confirming party.
FR-20	No settlement until cash-in is confirmed.	Settlement is enqueued only inside <code>confirm-cash-in</code> ; no Payment is submitted before it.

5.6 Queue and settlement

ID	Requirement	Implemented in
FR-21	Queue a settlement message once cash-in is confirmed.	Confirm-cash-in writes a <code>SettlementMessage</code> row (outbox) and adds it to the Redis Stream <code>settlement_messages</code> .
FR-22	A worker consumes messages and submits the transfer to XRPL Testnet.	<code>scripts/run_settlement_worker.py</code> or POST <code>/admin/settlement/run</code> reads the stream as a consumer group and submits a UCTUSD Payment with <code>xrpl-py</code> . Each call drains the stream once; the worker does not stay running between calls.
FR-23	Record each outcome, with the transaction hash on success.	Hash or failure reason stored on the remittance and the message; GET <code>/admin/settlement</code> ; UI <code>/admin/settlement</code> .
FR-24	A failed transaction does not credit the recipient.	Status becomes <code>settlement_failed</code> with no credit; covered by <code>tests/test_settlement.py</code> .
FR-25	A duplicate message cannot credit twice.	One message per remittance (unique constraint); if a hash is already stored the worker marks the job complete without paying or crediting; retrying a completed message returns 409. Before submitting, the worker looks up a validated Payment whose memo is this remittance id and adopts that hash instead of paying again. Two overlapping worker runs can still both miss the lookup and pay twice.
FR-26	Update the wallet balance once settlement is confirmed on-ledger.	The worker waits for ledger validation, then stores the hash, credits the balance and sets <code>settled</code> in one database commit. If the process dies before that commit, the next run adopts the payment by its memo and then credits the wallet.

5.7 Recipient wallet and sender history

ID	Requirement	Implemented in
FR-27	Each recipient has a custodial web wallet.	A custodial Testnet account per recipient, secret encrypted with its own Fernet key; GET <code>/wallet/me</code> ; UI <code>/app/wallet</code> .
FR-28	Wallet shows balance, incoming transfers, cash-outs, status, date and transaction hash.	Spendable and on-chain balances; incoming transfers with amount, status, dates and hash; cash-outs with amount, currency, status, dates and burn hash. The on-chain figure is spendable plus requested or approved cash-out amounts; it is not a live read from the ledger.
FR-28a	Sender can view status and history of sent remittances.	GET <code>/remittances/me</code> lists amount, status and tracking reference on <code>/app/history</code> . The quote breakdown, status timeline, hash and a scannable QR code are on <code>/app/track/{ref}</code> (GET <code>/remittances/track/{ref}</code>) and the quote page.

5.8 Simulated cash-out

ID	Requirement	Implemented in
FR-29	Establish a TrustLine to the issuer for any wallet holding the token.	TrustSet for the platform treasury (<code>scripts/setup_platform_wallet.py</code> or POST <code>/admin/platform-wallet/trustline</code>) and for each recipient account before its first payment.
FR-30	Recipient requests a cash-out to USD or local fiat.	POST <code>/cash-outs</code> to USD or ZAR; the amount is reserved from the spendable balance at once.
FR-31	Payout uses the applicable rate, less the cash-out fee.	Fee taken from the UCTUSD amount; USD pays out 1:1, ZAR at the live or fallback USD/ZAR rate without the FX margin.
FR-32	Track cash-outs as requested, approved, completed or failed.	requested → approved → completed, or failed from requested or approved. Complete burns the gross reserved UCTUSD by paying the issuer; the fiat payout is the net after the fee and no bank payment is made. Fail refunds the gross amount to the receiver's wallet.

5.9 Administration

ID	Requirement	Implemented in
FR-33	Admin interface for KYC applications.	UI <code>/admin/kyc</code> (FR-07, FR-08).
FR-34	Admin interface for confirming cash-in.	UI <code>/admin/cash-in</code> (FR-19).
FR-35	Admin interface for actioning cash-outs.	UI <code>/admin/cash-out</code> : POST <code>/cash-outs/{id}/approve</code> , <code>/complete</code> , <code>/fail</code> . Fail refunds the cash-out user (the recipient), not the remittance sender. Each action is confirmed in a dialog.

FR-15a and FR-16b are also administration functions; they are listed with the fees and limits they configure. Beyond the requirements, the system adds a 15-minute quote expiry with cancel (`POST /remittances/{id}/cancel`), MG tracking references, admin settlement retry, an FAQs page and a platform-wallet screen reporting treasury TrustLine and liquidity.

6 Fee Model

Several fees are added to transactions. These include a transaction fee when the sender pays in rand, a margin on the exchange rate, and a fee when the recipient cashes out. The amounts for these fees are stored in a single settings record, `FeeConfig`. This enables an administrator to change them on the Config screen after logging in as an admin, and the next quote uses the new values, with no code change or restart. Quotes already issued are unaffected, because each quote stores the fee and rate it was priced at.

The baseline system starts with a fixed fee of R25.00, a percentage fee of 1% of the amount sent, an FX margin of 2%, and a cash-out fee of 1.5%. Percentages are stored as fractions, so 1% is stored as 0.0100.

The transaction fee is the fixed fee plus a percentage of the amount sent (`zar_amount`), and it is deducted from the rand amount first. The remainder (`net_zar`) is then converted to dollars at the `effective_rate` (mid-market rate, marked up by the `fx_margin`). The resulting amount (`rlusd`) is the amount of UCTUSD the recipient receives:

$$\begin{aligned}
 \text{transaction_fee} &= \text{fixed fee} + \text{zar_amount} \times \text{percentage} \\
 \text{effective_rate} &= \text{midmarket} \times (1 + \text{fx_margin}) \\
 \text{net_zar} &= \text{zar_amount} - \text{transaction_fee} \\
 \text{rlusd} &= \text{net_zar} / \text{effective_rate}
 \end{aligned}$$

Since the margin increases the rand amount needed for each dollar, the receiver receives fewer dollars than the mid-market rate would provide. The difference between the mid-market and effective rate is the platform's exchange-rate income, and this margin is shown in the quote. The token is treated as being worth exactly one US dollar. The rand transaction fee and the fiat payout are rounded to two decimal places. The token amounts and the cash-out fee are rounded to six. Halves are rounded up. If the fee is as large as the amount sent, then the

quote is refused (with an HTTP 422 error), since no money is left to convert.

The cash-out fee on the quote is only an estimate, as the recipient may cash out in part, at a later time or in a different currency. When they do, the cash-out fee is subtracted from the token amount using the cash-out fee set at that time. If a USD cash-out is selected then one dollar per token is paid. If a ZAR cash-out is selected then a conversion is performed at the current USD/ZAR rate, or the fallback rate, with no FX margin added.

A worked example can be seen below:

Step (R1,000 at the 18.50 fallback rate)	Value
Transaction fee	R35.00
Effective rate	18.87
Net rand	R965.00
UCTUSD to recipient	51.139375
Estimated cash-out fee	0.767091
Estimated payout (UCTUSD)	50.372284

Table 11: Worked quote at the default fees.

7 Exchange-Rate Calculation

The platform uses a public API to retrieve exchange rates, with a configured rate as the backup.

When a quote is created, the system requests the latest US dollar rates from a free public service (open.er-api.com) and retrieves the rand rate from the reply. This rate is retained for five minutes and is reused for every quote made in that period, such that a burst of users does not flood the free service with requests. For every first quote after each five-minute period ends, a new rate is fetched.

If the exchange rate request fails for reasons such as a timeout, an error response, or a reply with no rand rate, the system uses a fixed backup rate ($\text{USD_ZAR_RATE} = 18.50$), which is configured in the settings file and keeps it for five minutes before trying the live service again.

The backup rate can only be changed in the settings file and not the admin screens. For backup rate changes to reflect the server needs to be restarted. The rate used, with the FX margin applied, is stored on each quote, so the sender sees the rate they were actually charged.

The automated tests conducted never contact the live service and instead substitute a fixed or simulated rate to ensure that test results are reproducible.

8 Remittance Limits

Remittance limits depend on whether the user's KYC is verified or unverified:

Tier	Daily limit	Monthly limit
Unverified	R0	R0
Verified	R3,000	R25,000

Table 12: Default limit tiers.

Admins may change these limits via the configurations tab. Unverified senders are refused earlier, with a 403 for missing KYC, so they never reach the R0 check; R0 is the configured tier shown on `GET /limits/me`.

Usage sums the sender's ZAR since the start of the UTC day or calendar month, with two exclusions:

1. Cancelled requests never count
2. `quoted` rows with `expires_at` already in the past never count.

A quote is refused only when usage plus the new amount is strictly over the cap, so a send that lands exactly on the cap is allowed.

Everything else counts towards the usage amount, including unexpired quotes, and every status from cash-in onwards, including `settlement_failed`. This is because a failed on-chain payment still follows a confirmed cash-in, so the sender's rand was received, and the amount stays counted. An administrator can retry the settlement, but nothing releases the amount early. Thus, the usage sum only stops counting when the day or month ends.

Status	How it gets there	Counts to limit?
quoted	POST /remittances	Yes, until TTL
cancelled	Sender cancels while still unexpired	No
quoted, after expiry	Cash-in returns 409 once <code>expires_at</code> has passed	No
cash_in_pending	Sender cash-in	Yes
cash_in_confirmed	Admin confirms	Yes
settlement_queued	Enqueued after confirm	Yes
settled	Worker Payment and credit	Yes
settlement_failed	Worker error; no credit	Yes

Table 13: Remittance statuses on one evolving row.

9 Architecture

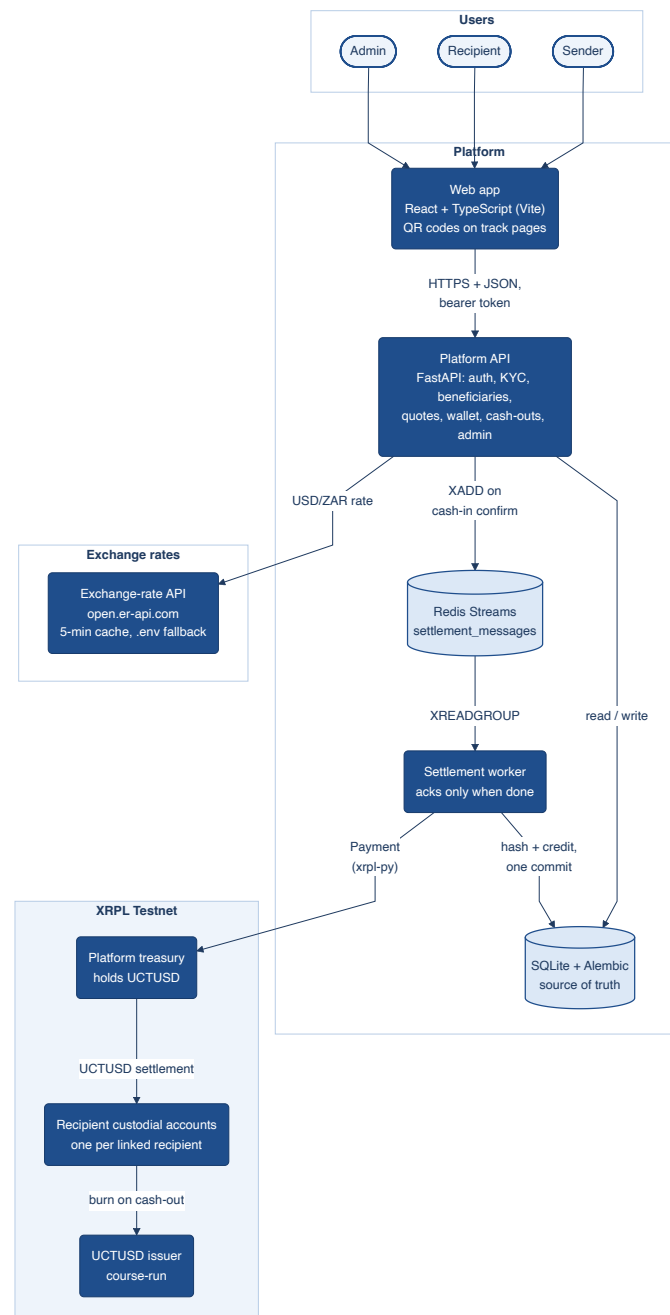


Figure 4: System architecture.

Users reach one React web app, which calls the FastAPI platform API with a bearer token. The API reads the live USD/ZAR rate, stores all state in SQLite, and on a confirmed cash-in adds a message to a Redis stream. The settlement worker reads that stream and pays UCTUSD from the platform treasury to the recipient's custodial Testnet account, then records the hash and credits the balance in one commit. The prototype uses both wallet options in the brief: a real Testnet account per linked recipient, and an internal spendable balance. The on-chain figure shown in the wallet is not a live ledger read.

10 Cash-in Flow

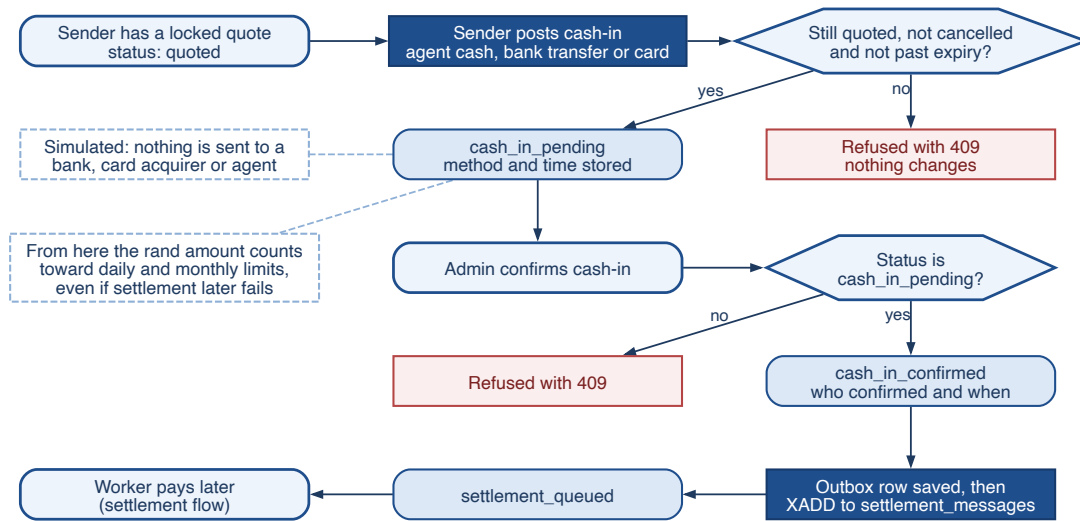


Figure 5: Cash-in flow.

Cash-in starts from a quote that is still quoted, not cancelled, and not past its expiry. The sender posts one method: agent cash, bank transfer, or card. The remittance stores that label and the time, and moves to cash-in pending. The method is not sent to a bank, a card acquirer, or an agent. No settlement message is created at this step, and no XRP Ledger Payment is submitted.

Only an administrator can confirm the cash-in, and only from cash-in pending. Confirmation records who confirmed it and when. It is the stand-in for a payment service saying the rand was received. Settlement is enqueued only inside that confirmation. The remittance then becomes settlement queued. Confirming does not itself submit the Payment. The worker does that later, when it reads the queue.

From cash-in pending onward the rand amount keeps counting toward the sender's daily and monthly limits, including if settlement later fails. A cancelled quote cannot be cashed in. An expired quote cannot be cashed in, and cash-in on it returns 409.

11 RLUSD Settlement Flow

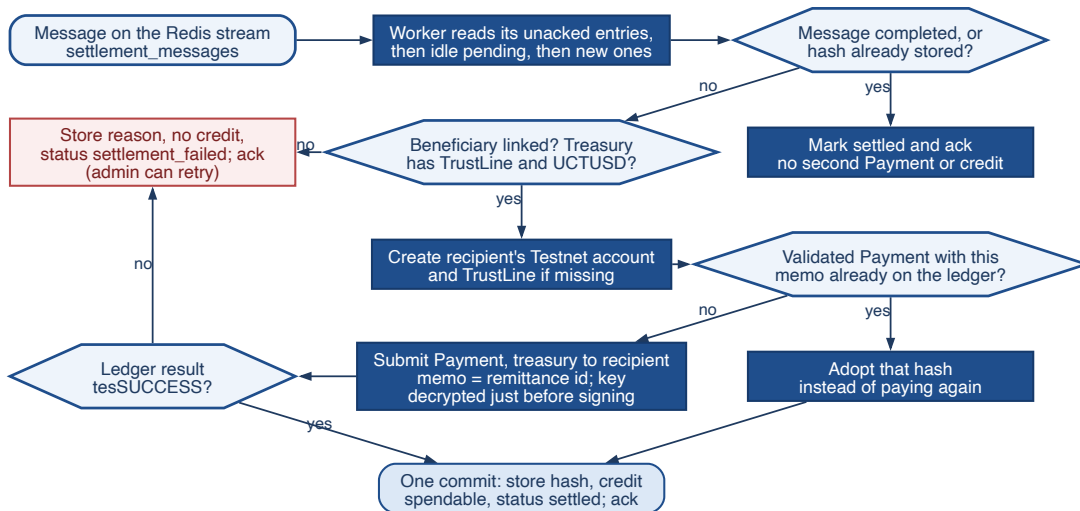


Figure 6: RLUSD settlement flow.

The brief calls this the RLUSD settlement flow. The token the worker actually pays is UCTUSD, issued by the configured Testnet account. Quote and wallet fields are still named with `rlusd`. That name is the amount. The ledger payment uses the configured UCTUSD currency code.

Confirming cash-in inserts one settlement message per remittance and publishes its identifier on the Redis stream `settlement_messages`. If that row already exists but the stream entry was never stored, the same row is published again, so a Redis failure after the database commit does not leave the remittance stranded. The worker reads its own unacknowledged entries, then idle entries waiting on the pending list, then new entries. It acknowledges an entry only after the message is completed or failed. A crash before that leaves the entry to be picked up again.

The worker will not pay unless the beneficiary is linked to a registered user, the platform wallet has a TrustLine to the UCTUSD issuer, and the treasury holds enough UCTUSD. It creates the recipient's Testnet account and TrustLine if they are not there yet, then submits a Payment of the quoted amount from the treasury to that account. The remittance identifier is the Payment memo. The ledger result must be `tesSUCCESS`.

On success the spendable balance is increased, the remittance is marked settled, and the transaction hash is stored, in one database commit. On failure the balance is not increased and the reason is stored on the message and the remittance. A message already completed, or a remittance that already has a hash, is not paid or credited again. An administrator can retry a pending, processing, or failed message by putting a new entry on the stream. A completed settlement cannot be retried. Before submitting, the worker looks up a validated Payment on the sending account whose memo is this remittance id and adopts that hash instead of paying again; the wallet is still credited if this process had not credited it yet. Two overlapping worker runs can still both miss the lookup and pay twice.

12 Cash-out Flow

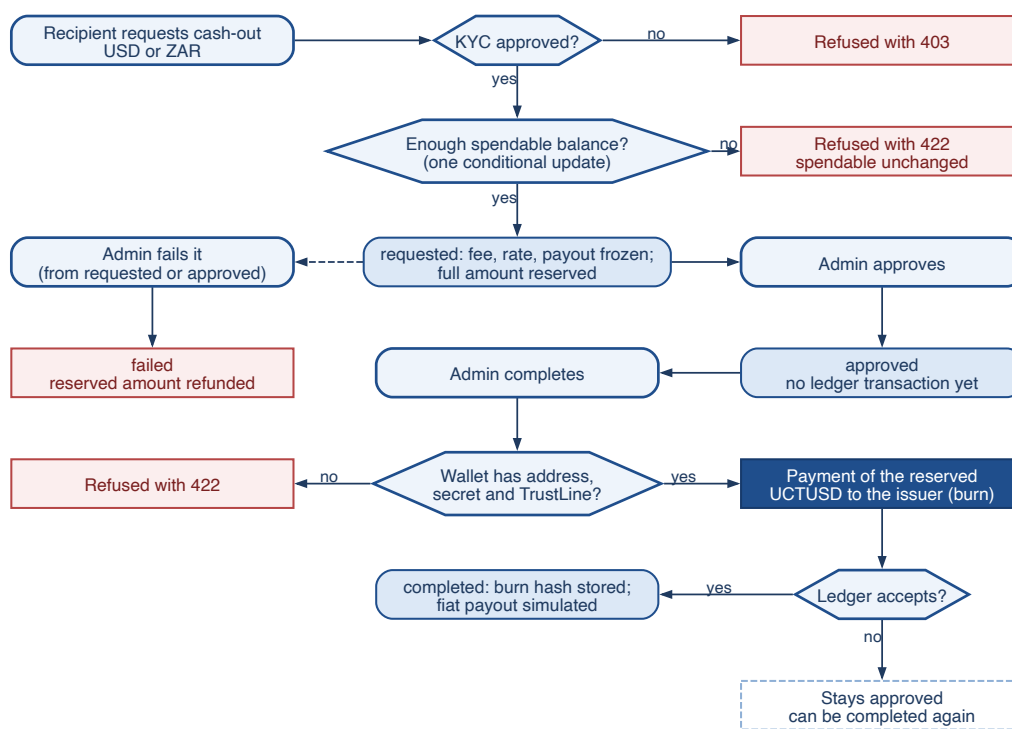


Figure 7: Cash-out flow.

A recipient can hold UCTUSD without approved KYC, but requesting a cash-out requires it. The request freezes the fee, the rate and the fiat payout on the cash-out row, and reserves the full token amount by reducing spendable in one conditional update. If the balance is too small, the request is refused and spendable is unchanged. USD pays one dollar per token after the cash-out fee. Rand uses the live or fallback USD/ZAR rate with no FX margin. Those figures are not recalculated when an administrator later approves or completes the request.

The wallet shows two numbers. Spendable is the internal balance after the reservation. The on-chain figure is not read live from the XRP Ledger. It is spendable plus the token amounts on this user's cash-outs that are still requested or approved, which are the tokens still sitting in the custodial account. After a failure, spendable is

PLATFORM_WALLET			
string	id	PK	
string	classic_address	UK	treasury account
string	secret		Fernet-encrypted
string	network		testnet
bool	trustline_established		

FEE_CONFIG			
string	id	PK	
decimal	fixed_fee_zar		25.00
decimal	percentage_fee		0.0100
decimal	fx_margin_percentage		0.0200
decimal	cash_out_fee_percentage		0.0150

LIMIT_TIERS			
string	id	PK	
enum	tier_key	UK	unverified verified
decimal	daily_limit_zar		
decimal	monthly_limit_zar		

Figure 9: Configuration tables.

14 API Overview

The API is JSON over HTTP. Register and login do not need an existing session. Login returns an opaque bearer token, its expiry, and the user. Later calls send that token as **Authorization: Bearer**. The database stores only the SHA-256 hash of the token. Logout revokes that session immediately, so the same token stops working. Interactive documentation is at `/docs`, and `GET /health` does not require a login.

Customers use the API for their own profile, KYC, beneficiaries, quotes, cash-in, cancellation, history, limits, wallet and cash-out requests. Adding a beneficiary, creating a quote and requesting a cash-out require approved KYC. Administrators review KYC, confirm cash-in, approve, complete or fail cash-outs, edit fees and limit tiers, inspect the platform wallet, and run or retry settlement. Both roles sign in through the same login, and each route checks which one is calling.

Looking up a remittance by its MG tracking reference is not public. The sender, the linked recipient or an administrator may see it; any other signed-in user is refused. Senders also have a list of the remittances they created. A linked recipient sees incoming transfers on the wallet rather than on that send history.

15 Security Design

Passwords are stored as bcrypt hashes and are never returned. Login creates an unguessable bearer token and stores only its SHA-256 hash. The raw token is handed back once. A session lasts twelve hours unless a user logs out which terminates the session sooner. Logout sets the time the session was revoked, and a later call with that token is rejected. Other sessions for the same person stay valid until they expire or are logged out.

Two separate encryption keys are read from the environment, not from the database. One encrypts the KYC identification number. The other encrypts the platform and recipient XRP Ledger secrets. A leak of one key does not decrypt the other enhancing Baobab's security. Wallet and platform-wallet responses include addresses and balances and do not include secrets. The row loads ciphertext and the seed is decrypted only when settlement, cash-out completion, or a TrustSet is about to sign. The identification number is decrypted when a KYC record is read, so that the applicant and an administrator reviewing the application can see it.

Validation checks are performed on mobile numbers which need to be 8 to 15 digits, and identification numbers as 6 to 20 characters (to meet international standards). However, these format checks are not a one-time password, and are not a check against Home Affairs or a passport authority. Approval is still a separate decision made by an administrator.

16 Regulatory Considerations

Settling transfers in RLUSD rather than through banking rails does not remove a remittance platform from the scope of South African financial regulation. The most relevant areas are customer verification and anti-money-laundering controls, licensing, exchange control, consumer protection and the safekeeping of customer assets.

16.1 KYC and AML

Since December 2022, any company providing crypto-asset services in South Africa has counted as an accountable institution under the Financial Intelligence Centre Act (FICA), placing them under the same obligations as

banks (Republic of South Africa, 2022). In practice, that means verifying who a customer is before any money moves, watching transactions for suspicious activity and reporting suspicious or large transfers to the Financial Intelligence Centre. Extending these obligations to crypto-asset firms formed part of South Africa's response to weaknesses identified by the Financial Action Task Force, which placed the country on its grey list from 2023 to 2025, and supervisory attention in this area remains high (Financial Action Task Force, 2025).

The platform's KYC process collects identity, address and source-of-funds information and allows only KYC-approved users to send money; an administrator must approve each submission. Daily and monthly limits on the rand amount a user sends are also enforced. These controls are not a transaction-monitoring or reporting engine: nothing screens sanctions lists or files reports with the Financial Intelligence Centre.

16.2 Licensing

In South Africa, any platform working with stablecoins and cross-border money transferring will require two forms of authorisation to operate.

The first concerns the crypto assets themselves. In October 2022, the Financial Sector Conduct Authority classified all crypto assets as financial products, so any company handling them on behalf of customers needs a licence under a dedicated crypto-assets category (Financial Sector Conduct Authority, 2022). Operating without one is a criminal offence, punishable by a fine of up to R10 million or up to ten years in prison (Republic of South Africa, 2002).

The second concerns moving money across the border, which the South African Reserve Bank regulates. Wise, for instance, was only able to start serving South African customers after the Reserve Bank conditionally approved it as an Authorised Dealer in Foreign Exchange with Limited Authority (Wise, 2025). The platform needs the same kind of licence, or a partnership with a dealer that already holds one. Holding that authorisation also means enforcing the Reserve Bank's limits on how much customers may send abroad. Each resident can currently send up to R2 million a year without special approval (South African Reserve Bank, 2026). Residency here depends on where a person lives, rather than on their nationality, so the allowance also applies to migrant workers who have settled in South Africa (FinGlobal, 2026). It is less clear whether these limits apply to crypto assets, since South African courts have reached conflicting decisions and new regulations covering crypto are still in draft (National Treasury, 2026). A licensed dealer would have to apply the annual allowance to these transfers. This prototype does not enforce it; it enforces only its own configurable daily and monthly caps (R3,000 and R25,000 for a verified sender).

16.3 Consumer protection

The General Code of Conduct under the Financial Advisory and Intermediary Services Act requires providers to act honestly, fairly, with due skill, care and diligence, and in the interests of their clients (Republic of South Africa, 2002). The platform applies this first to price. Senders see every fee, the exchange rate and an estimate of what their recipient will receive, before they confirm a transfer, rather than discovering part of the cost later in the rate. The cash-out fee and payout are estimates, recalculated when the recipient actually cashes out.

The same duty extends to personal information. The Financial Intelligence Centre Act requires providers such as the platform to collect identity numbers, addresses and source-of-funds details, while the Protection of Personal Information Act governs how that information is kept (Republic of South Africa, 2001). It must be secured against loss and unauthorised access, and it may only be sent outside South Africa under certain conditions, for example with the person's consent or where the recipient offers adequate data protection (Republic of South Africa, 2013).

16.4 Custody and safeguarding of customer assets

Because the platform holds the private keys to each recipient's custodial account, it carries responsibility for keeping those assets safe. In the prototype, one platform treasury pays each recipient's custodial Testnet account, and each user's spendable balance is an internal ledger. There is no separate bank account for client money, since cash-in and fiat payout are simulated. Private keys are stored encrypted, the decryption key is kept in the environment rather than the database, and a key is decrypted only immediately before settlement, cash-out completion or a TrustSet signs.

16.5 In the prototype

In the prototype, these duties appear only as the controls the software actually has: a KYC application, administrator approval before sending or cashing out, daily and monthly rand caps, and encrypted storage of the

identification number and of the ledger secrets. The software does not screen sanctions lists, does not file reports, and does not hold a licence. Cash-in and fiat cash-out are simulated, so the system does not take rand or pay out dollars.

17 Assumptions and Limitations

Both senders and receivers are registered as users. Administrators are created outside public registration. Beneficiaries are linked by matching mobile number or email, including when the recipient registers later. There is no invitation message. A person without approved KYC can register, hold UCTUSD and open the wallet, but cannot add a beneficiary, lock a quote or cash out. That block is a refusal for missing KYC, not a journey that reaches the unverified R0 cap and fails there.

The on-ledger asset is UCTUSD on the XRP Ledger Testnet, unless the issuer and currency code in configuration are changed. One token is treated as one US dollar in the fee arithmetic. Quotes expire after 900 seconds. Cash-in methods are labels only. Cash-out fiat is US dollars or rand only. The default database is SQLite, and Redis must be running for a confirmed cash-in to be picked up by the worker.

The daily and monthly check reads current usage and then inserts the quote. Nothing locks the sender's rows between those two steps, so two quotes submitted together can both pass and together exceed the cap. Settlement will not pay twice once the transaction hash has been saved, and before submitting, the worker adopts an earlier validated Payment with the same memo. Two overlapping worker runs can still both miss that lookup and pay twice. The automated tests replace the ledger calls, so they do not show a live Testnet payment but live testing of the platform confirmed that the logic is implemented correctly. There is also no sanctions or transaction-monitoring engine in the application.

References

- Financial Action Task Force (2025) *Jurisdictions under increased monitoring – 24 October 2025*. Paris: Financial Action Task Force.
- Financial Sector Conduct Authority (2022) *Declaration of a crypto asset as a financial product under the Financial Advisory and Intermediary Services Act, 2002*. General Notice 1350 of 2022. Pretoria: Financial Sector Conduct Authority.
- FinGlobal (2026) *Exchange control South Africa*. Available at: <https://www.finglobal.com/exchange-control-south-africa/>.
- FinMark Trust (2024) *South Africa to the rest of SADC remittances market assessment 2024 report*. Johannesburg: FinMark Trust.
- International Monetary Fund (2025) *How stablecoins can improve payments and global finance*. Available at: <https://www.imf.org/en/blogs/articles/2025/12/04/how-stablecoins-can-improve-payments-and-global-finance>.
- Katz, D. (2026) *Stablecoins: promise, risks, and policy choices for emerging markets*. Remarks at the University of Cape Town, 7 August. Available at: <https://www.imf.org/en/news/articles/2026/08/07/sp080726-stablecoins-emerging-markets-dan-katz>.
- National Treasury (2026) *Draft Capital Flow Management Regulations, 2026*. Government Notice No. 54520, Government Gazette No. 7375, 17 April.
- Republic of South Africa (2001) *Financial Intelligence Centre Act 38 of 2001*. Pretoria: Government Printer.
- Republic of South Africa (2002) *Financial Advisory and Intermediary Services Act 37 of 2002*. Pretoria: Government Printer.
- Republic of South Africa (2013) *Protection of Personal Information Act 4 of 2013*. Pretoria: Government Printer.
- Republic of South Africa (2022) *Amendment of Schedule 1 to the Financial Intelligence Centre Act 38 of 2001*. Government Gazette No. 47596, 29 November.
- Rey, H. (2025) 'Stablecoins, tokens, and global dominance', *Finance & Development*, 62(3). Available at: <https://www.imf.org/-/media/files/publications/fandd/article/2025/09/reypdf>.
- South African Reserve Bank (2026) *Exchange Control Circular No. 6/2026: single discretionary allowance*. Pretoria: South African Reserve Bank.
- United Nations (2015) *Transforming our world: the 2030 Agenda for Sustainable Development*. New York: United Nations.
- VALR (2025) *How to send money from South Africa to Zimbabwe (with screenshots)*. Available at: <https://blog.valr.com/blog/send-money-zimbabwe-south-africa>.
- Wise (2025) *Wise receives conditional regulatory approval to launch in South Africa, marking company's first license in Africa*. Available at: <https://newsroom.wise.com/en-CEU/258484-wise-receives-conditional-regulatory-approval-to-launch-in-south-africa-marking-company-s-first-license-in-africa/>.
- Wise (2026) *Wise Platform enters South Africa with Capitec to modernise international payments*. Available at: <https://www.wise.com/press/wise-platform-enters-south-africa-with-capitec-to-modernise-international-payments>.

[//newsroom.wise.com/en-CEU/264197-wise-platform-enters-south-africa-to-modernise-international-payments/](https://newsroom.wise.com/en-CEU/264197-wise-platform-enters-south-africa-to-modernise-international-payments/).

World Bank (2025) *Remittance Prices Worldwide: South Africa to Zimbabwe*. Available at: <https://remittanceprices.worldbank.org/corridor/South-Africa/Zimbabwe>.